

Mini RFC: QUIC Handshake

April 21, 2022

Note: This is a simplified and annotated summary of the QUIC-related IETF RFCs. It's part of a course project at Tsinghua University and not suitable for any other purposes.

We reorder paragraphs in QUIC RFCs and omit many details to improve its readability for beginners.

WARNING: DO NOT distribute this document. Modification of extracts from IETF RFCs is not granted by IETF as original authors retain copyrights of RFCs. This document is for educational purposes only.

1 Preliminary

QUIC embeds a (variant of) TLS 1.3 handshake in its own handshake. You have to learn how TLS handshake works to understand the cryptographic part of the QUIC handshake. Though it's not very necessary to learn the underlying mathematical theory such as Elliptic Curve, you should be familiar with cryptographic terminology such as cipher suite, key exchange, block cipher, mode of operation, public key infrastructure and so on. If you know what "TLS_AES_128_GCM_SHA256" means and what "public key", "private key", "public key certificate", "root certificate" are, you are prepared.

You could learn required knowledge of cryptography quickly by searching all unknown terminology by Google or Wikipedia.

2 Overview

This document describes how QUIC is secured using TLS [RFC 8446].

TLS 1.3 provides critical latency improvements for connection establishment over previous versions. Absent packet loss, most new connections can be established and secured within a single round trip; on subsequent connections between the same client and server, the client can often send application data immediately, that is, using a zero round-trip setup.

This document describes how TLS acts as a security component of QUIC.

3 TLS

TLS provides two endpoints with a way to establish a means of communication over an untrusted medium (for example, the Internet). TLS enables

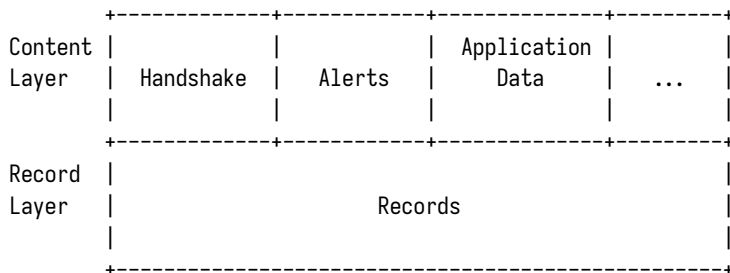


Figure 1: *TLS Layers*

authentication of peers and provides confidentiality and integrity protection for messages that endpoints exchange.

Internally, TLS is a layered protocol, with the structure shown in [Figure 1](#).

Each content-layer message (e.g., handshake, alerts, and application data) is carried as a series of typed TLS records by the record layer. Records are individually cryptographically protected and then transmitted over a reliable transport (typically TCP), which provides sequencing and guaranteed delivery.

The TLS authenticated key exchange occurs between two endpoints: client and server. The client initiates the exchange and the server responds. If the key exchange completes successfully, both client and server will agree on a secret. TLS supports both pre-shared key (PSK) and Diffie-Hellman over either finite fields or elliptic curves ((EC)DHE) key exchanges. PSK is the basis for Early Data (0-RTT); the latter provides forward secrecy (FS) when the (EC)DHE keys are destroyed. The two modes can also be combined to provide forward secrecy while using the PSK for authentication.

After completing the TLS handshake, the client will have learned and authenticated an identity for the server, and the server is optionally able to learn and authenticate an identity for the client. TLS supports X.509 [\[RFC 5280\]](#) certificate-based authentication for both server and client. When PSK key exchange is used (as in resumption), knowledge of the PSK serves to authenticate the peer.

The TLS key exchange is resistant to tampering by attackers, and it produces shared secrets that cannot be controlled by either participating peer.

TLS provides two basic handshake modes of interest to QUIC:

- A full 1-RTT handshake, in which the client is able to send application data after one round trip and the server immediately responds after receiving the first handshake message from the client.
- A 0-RTT handshake, in which the client uses information it has previously learned about the server to send application data immediately. This application data can be replayed by an attacker, so 0-RTT is not

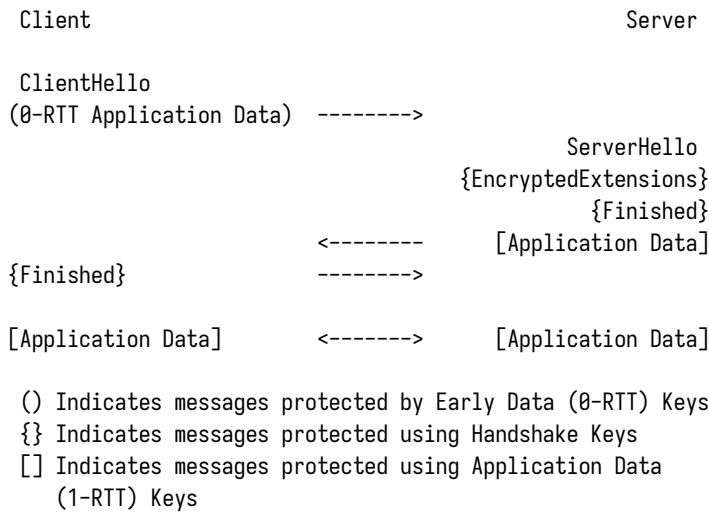


Figure 2: *TLS Handshake with 0-RTT*

suitable for carrying instructions that might initiate any action that could cause unwanted effects if replayed.

A simplified TLS handshake with 0-RTT application data is shown in **Figure 2**.

Data is protected using a number of encryption levels:

- Initial keys
- **Early data (0-RTT) keys**
- Handshake keys
- Application data (1-RTT) keys

Application data can only appear in the early data and application data levels. Handshake and alert messages may appear in any level.

The 0-RTT handshake can be used if the client and server have previously communicated. In the 1-RTT handshake, the client is unable to send protected application data until it has received all of the handshake messages sent by the server.

4 Protocol Overview

QUIC assumes responsibility for the confidentiality and integrity protection of packets. For this it uses keys derived from a TLS handshake, but instead of carrying TLS records over QUIC (as with TCP), TLS handshake and alert messages are carried directly over the QUIC transport, which takes over the responsibilities of the TLS record layer, as shown in **Figure 3**.

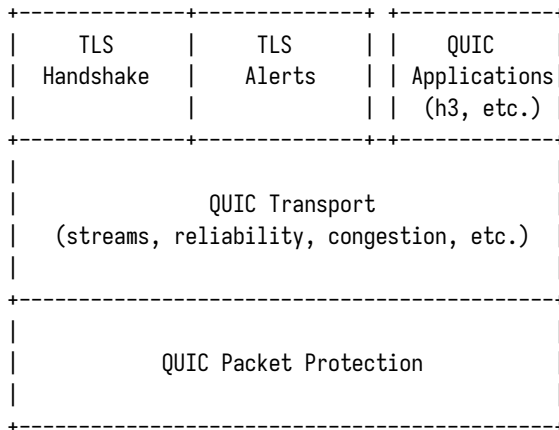


Figure 3: *QUIC Layers*

QUIC also relies on TLS for authentication and negotiation of parameters that are critical to security and performance.

Rather than a strict layering, these two protocols cooperate: QUIC uses the TLS handshake; TLS uses the reliability, ordered delivery, and record layer provided by QUIC.

At a high level, there are two main interactions between the TLS and QUIC components:

- The TLS component sends and receives messages via the QUIC component, with QUIC providing a reliable stream abstraction to TLS.
- The TLS component provides a series of updates to the QUIC component, including (a) new packet protection keys to install and (b) state changes such as handshake completion, the server certificate, etc.

Figure 4 shows these interactions in more detail, with the QUIC packet protection being called out specially.

Unlike TLS over TCP, QUIC applications that want to send data do not send it using TLS Application Data records. Rather, they send it as QUIC STREAM frames or other frame types, which are then carried in QUIC packets.

If we build TLS atop of QUIC, like how TLS is integrated with TCP, we could not protect the QUIC header using TLS. Or to say, QUIC should not be TLS-agnostic.

TLS 1.3 is designed and standardized simultaneously with QUIC.

5 Packet Number Space

Packet numbers are divided into three spaces in QUIC:

- Initial space: All Initial packets are in this space.
- Handshake space: All Handshake packets are in this space.
- Application data space: All 0-RTT and 1-RTT packets are in this space.

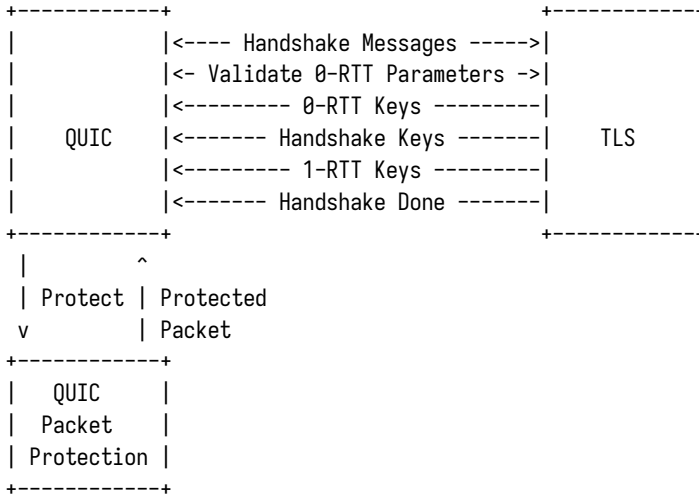


Figure 4: *QUIC and TLS Interactions*

Each packet type uses different protection keys.

Conceptually, a packet number space is the context in which a packet can be processed and acknowledged. Initial packets can only be sent with Initial packet protection keys and acknowledged in packets that are also Initial packets. Similarly, Handshake packets are sent at the Handshake encryption level and can only be acknowledged in Handshake packets.

This enforces cryptographic separation between the data sent in the different packet number spaces. Packet numbers in each space start at packet number 0. Subsequent packets sent in the same packet number space **MUST** increase the packet number by at least one.

0-RTT and 1-RTT data exist in the same packet number space to make loss recovery algorithms easier to implement between the two packet types.

A QUIC endpoint **MUST NOT** reuse a packet number within the same packet number space in one connection. If the packet number for sending reaches $2^{62} - 1$, the sender **MUST** close the connection without sending a `CONNECTION_CLOSE` frame or any further packets; **an endpoint MAY send a Stateless Reset in response to further packets that it receives.**

A receiver **MUST** discard a newly unprotected packet unless it is certain that it has not processed another packet with the same packet number from the same packet number space.

Some frames are prohibited in different packet number spaces. The rules here generalize those of TLS, in that frames associated with establishing the connection can usually appear in packets in any packet number space, whereas those associated with transferring data can only appear in the application data packet number space:

- `PADDING`, `PING`, and `CRYPTO` frames **MAY** appear in any packet number space.

- CONNECTION_CLOSE frames signaling errors at the QUIC layer (type 0x1c) MAY appear in any packet number space. CONNECTION_CLOSE frames signaling application errors (type 0x1d) MUST only appear in the application data packet number space.
- ACK frames MAY appear in any packet number space but can only acknowledge packets that appeared in that packet number space. However, as noted below, 0-RTT packets cannot contain ACK frames.
- All other frame types MUST only be sent in the application data packet number space.

6 Cryptographic Handshake

QUIC relies on a combined cryptographic and transport handshake to minimize connection establishment latency. QUIC uses the CRYPTO frame to transmit the cryptographic handshake. The version of QUIC defined in this document is identified as 0x00000001; a different QUIC version could indicate that a different cryptographic handshake protocol is in use.

QUIC provides reliable, ordered delivery of the cryptographic handshake data. QUIC packet protection is used to encrypt as much of the handshake protocol as possible. The cryptographic handshake MUST provide the following properties:

- authenticated key exchange, where
 - a server is always authenticated,
 - a client is optionally authenticated,
 - every connection produces distinct and unrelated keys, and
 - keying material is usable for packet protection for both 0-RTT and 1-RTT packets.
- authenticated exchange of values for transport parameters of both endpoints, and confidentiality protection for server transport parameters.
- authenticated negotiation of an application protocol (TLS uses Application-Layer Protocol Negotiation (ALPN) [ALPN] for this purpose).

6.1 QUIC Handshake

The CRYPTO frame can be sent in different packet number spaces. The offsets used by CRYPTO frames to ensure ordered delivery of cryptographic handshake data start from zero in each packet number space.

Figure 5 shows a simplified handshake and the exchange of packets and frames that are used to advance the handshake. Exchange of application data during the handshake is enabled where possible, shown with an asterisk (“*”). Once the handshake is complete, endpoints are able to exchange application data freely.

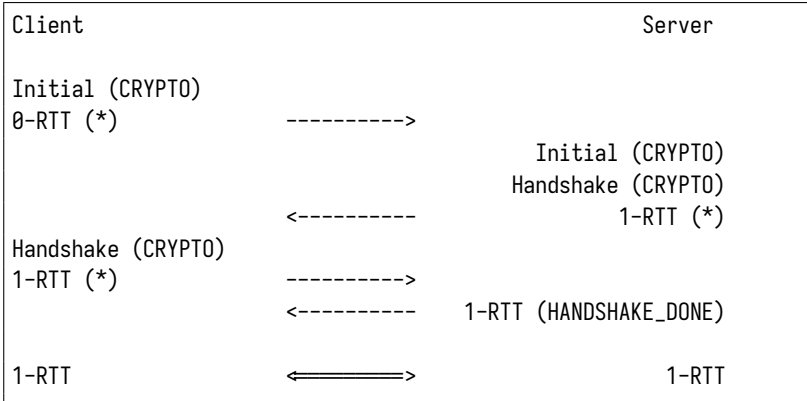


Figure 5: *Simplified QUIC Handshake*

Endpoints **MUST** explicitly negotiate an application protocol. This avoids situations where there is a disagreement about the protocol that is in use.

The cryptographic handshake is carried in Initial and Handshake packets.

Figure 6 provides an overview of the 1-RTT handshake. Each line shows a QUIC packet with the packet type and packet number shown first, followed by the frames that are typically contained in those packets. For instance, the first packet is of type Initial, with packet number 0, and contains a CRYPTO frame carrying the ClientHello.

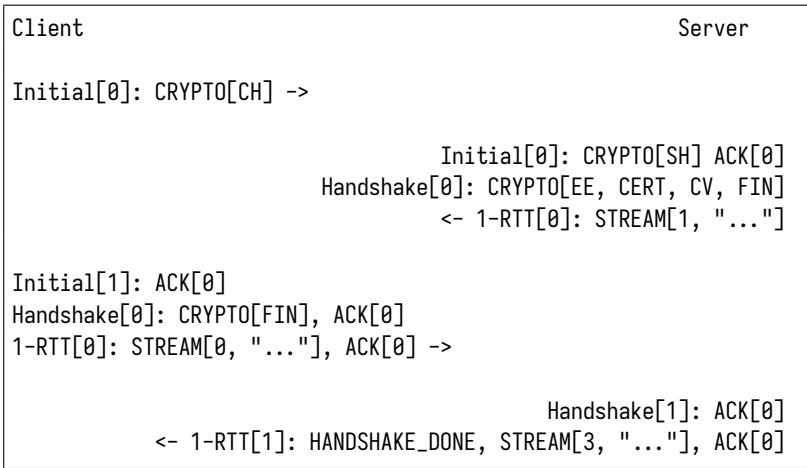


Figure 6: *Example 1-RTT Handshake*

6.2 Cryptographic Message Buffering

Implementations need to maintain a buffer of CRYPTO data received out of order. Because there is no flow control of CRYPTO frames, an endpoint could potentially force its peer to buffer an unbounded amount of data.

Implementations **MUST** support buffering at least 4096 bytes of data received in out-of-order CRYPTO frames. Endpoints **MAY** choose to allow more data to be buffered during the handshake. A larger limit during the handshake could allow for larger keys or credentials to be exchanged. An endpoint's buffer size does not need to remain constant during the life of the connection.

Being unable to buffer CRYPTO frames during the handshake can lead to a connection failure. If an endpoint's buffer is exceeded during the handshake, it can expand its buffer temporarily to complete the handshake. **If an endpoint does not expand its buffer, it **MUST** close the connection with a CRYPTO_BUFFER_EXCEEDED error code.**

Once the handshake completes, if an endpoint is unable to buffer all data in a CRYPTO frame, it **MAY discard that CRYPTO frame and all CRYPTO frames received in the future, or it **MAY** close the connection with a CRYPTO_BUFFER_EXCEEDED error code.** Packets containing discarded CRYPTO frames **MUST** be acknowledged because the packet has been received and processed by the transport even though the CRYPTO frame was discarded.

6.3 Carrying TLS Messages

QUIC carries TLS handshake data in CRYPTO frames, each of which consists of a contiguous block of handshake data identified by an offset and length. Those frames are packaged into QUIC packets and encrypted under the current encryption level. As with TLS over TCP, once TLS handshake data has been delivered to QUIC, it is QUIC's responsibility to deliver it reliably. Each chunk of data that is produced by TLS is associated with the set of keys that TLS is currently using. If QUIC needs to retransmit that data, it **MUST** use the same keys even if TLS has already updated to newer keys.

Each encryption level corresponds to a packet number space. The packet number space that is used determines the semantics of frames. Some frames are prohibited in different packet number spaces.

As shown in **Figure 4**, the interface from QUIC to TLS consists of four primary functions:

- Sending and receiving handshake messages
- **Processing stored transport and application state from a resumed session and determining if it is valid to generate or accept 0-RTT data**
- **Rekeying (both transmit and receive)**
- Updating handshake state

Additional functions might be needed to configure TLS. In particular, QUIC and TLS need to agree on which is responsible for validation of peer credentials, such as certificate validation [RFC 5280].

6.3.1 Handshake Complete & Confirmed

In this document, the TLS handshake is considered complete when the TLS stack has reported that the handshake is complete. This happens when the TLS stack has both sent a Finished message and verified the peer's Finished message. Verifying the peer's Finished message provides the endpoints with an assurance that previous handshake messages have not been modified. Note that the handshake does not complete at both endpoints simultaneously. Consequently, any requirement that is based on the completion of the handshake depends on the perspective of the endpoint in question.

In this document, the TLS handshake is considered confirmed at the server when the handshake completes. The server **MUST** send a HANDSHAKE_DONE frame as soon as the handshake is complete. At the client, the handshake is considered confirmed when a HANDSHAKE_DONE frame is received.

Additionally, a client **MAY** consider the handshake to be confirmed when it receives an acknowledgment for a 1-RTT packet. This can be implemented by recording the lowest packet number sent with 1-RTT keys and comparing it to the Largest Acknowledged field in any received 1-RTT ACK frame: once the latter is greater than or equal to the former, the handshake is confirmed.

6.3.2 Sending and Receiving Handshake Messages

In order to drive the handshake, TLS depends on being able to send and receive handshake messages. There are two basic functions on this interface: one where QUIC requests handshake messages and one where QUIC provides bytes that comprise handshake messages.

Before starting the handshake, QUIC provides TLS with the transport parameters that it wishes to carry.

A QUIC client starts TLS by requesting TLS handshake bytes from TLS. The client acquires handshake bytes before sending its first packet. A QUIC server starts the process by providing TLS with the client's handshake bytes.

At any time, the TLS stack at an endpoint will have a current sending encryption level and a receiving encryption level. TLS encryption levels determine the QUIC packet type and keys that are used for protecting data.

Each encryption level is associated with a different sequence of bytes, which is reliably transmitted to the peer in CRYPTO frames. When TLS provides handshake bytes to be sent, they are appended to the handshake bytes for the current encryption level. The encryption level then determines the type of packet that the resulting CRYPTO frame is carried in.

Four encryption levels are used, producing keys for Initial, 0-RTT, Handshake, and 1-RTT packets. CRYPTO frames are carried in just three of these levels,

omitting the 0-RTT level. These four levels correspond to three packet number spaces: Initial and Handshake encrypted packets use their own separate spaces; 0-RTT and 1-RTT packets use the application data packet number space.

QUIC takes the unprotected content of TLS handshake records as the content of CRYPTO frames. TLS record protection is not used by QUIC. QUIC assembles CRYPTO frames into QUIC packets, which are protected using QUIC packet protection.

QUIC CRYPTO frames only carry TLS handshake messages. TLS alerts are turned into QUIC CONNECTION_CLOSE error codes. TLS application data and other content types cannot be carried by QUIC at any encryption level; it is an error if they are received from the TLS stack.

When an endpoint receives a QUIC packet containing a CRYPTO frame from the network, it proceeds as follows:

- If the packet uses the current TLS receiving encryption level, sequence the data into the input flow as usual. As with STREAM frames, the offset is used to find the proper location in the data sequence. If the result of this process is that new data is available, then it is delivered to TLS in order.
- If the packet is from a previously installed encryption level, it **MUST NOT** contain data that extends past the end of previously received data in that flow. **Implementations MUST treat any violations of this requirement as a connection error of type `PROTOCOL_VIOLATION`.**
- If the packet is from a new encryption level, it is saved for later processing by TLS. Once TLS moves to receiving from this encryption level, saved data can be provided to TLS. **When TLS provides keys for a higher encryption level, if there is data from a previous encryption level that TLS has not consumed, this MUST be treated as a connection error of type `PROTOCOL_VIOLATION`.**

We are very strict with the handshake message as it's critical to security.

Each time that TLS is provided with new data, new handshake bytes are requested from TLS. TLS might not provide any bytes if the handshake messages it has received are incomplete or it has no data to send.

The content of CRYPTO frames might either be processed incrementally by TLS or buffered until complete messages or flights are available. TLS is responsible for buffering handshake bytes that have arrived in order. QUIC is responsible for buffering handshake bytes that arrive out of order or for encryption levels that are not yet ready. QUIC does not provide any means of flow control for CRYPTO frames.

Once the TLS handshake is complete, this is indicated to QUIC along with any final handshake bytes that TLS needs to send. **At this stage, the transport parameters that the peer advertised during the handshake are authenticated.**

Once the handshake is complete, TLS becomes passive. TLS can still receive data from its peer and respond in kind, but it will not need to send

more data unless specifically requested – either by an application or QUIC. One reason to send data is that the server might wish to provide additional or updated session tickets to a client.

When the handshake is complete, QUIC only needs to provide TLS with any data that arrives in CRYPTO streams. In the same manner that is used during the handshake, new data is requested from TLS after providing received data.

6.3.3 Encryption Level Changes

As keys at a given encryption level become available to TLS, TLS indicates to QUIC that reading or writing keys at that encryption level are available.

The availability of new keys is always a result of providing inputs to TLS. TLS only provides new keys after being initialized (by a client) or when provided with new handshake data.

However, a TLS implementation could perform some of its processing asynchronously. In particular, the process of validating a certificate can take some time. While waiting for TLS processing to complete, an endpoint SHOULD buffer received packets if they might be processed using keys that are not yet available. These packets can be processed once keys are provided by TLS. An endpoint SHOULD continue to respond to packets that can be processed during this time.

After processing inputs, TLS might produce handshake bytes, keys for new encryption levels, or both.

TLS provides QUIC with three items as a new encryption level becomes available:

- A secret
- An Authenticated Encryption with Associated Data (AEAD) function
- A Key Derivation Function (KDF)

These values are based on the values that TLS negotiates and are used by QUIC to generate packet and header protection keys.

Although TLS only uses one encryption level at a time, QUIC may use more than one level. For instance, after sending its Finished message (using a CRYPTO frame at the Handshake encryption level) an endpoint can send STREAM data (in 1-RTT encryption). If the Finished message is lost, the endpoint uses the Handshake encryption level to retransmit the lost message. Reordering or loss of packets can mean that QUIC will need to handle packets at multiple encryption levels. During the handshake, this means potentially handling packets at higher and lower encryption levels than the current encryption level used by TLS.

In particular, server implementations need to be able to read packets at the Handshake encryption level at the same time as the 0-RTT encryption level. A client could interleave ACK frames that are protected with

Handshake keys with 0-RTT data, and the server needs to process those acknowledgments in order to detect lost Handshake packets.

Figure 7 summarizes the exchange between QUIC and TLS for both client and server. Solid arrows indicate packets that carry handshake data; dashed arrows show where application data can be sent. Each arrow is tagged with the encryption level used for that transmission.

Figure 7 shows one possible structure for a simple handshake exchange. The exact process varies based on the structure of endpoint implementations and the order in which packets arrive. Implementations could use a different number of operations or execute them in other orders.

You could refer to [\[RFC 8446\]](#) for standard TLS 1.3 handshake.

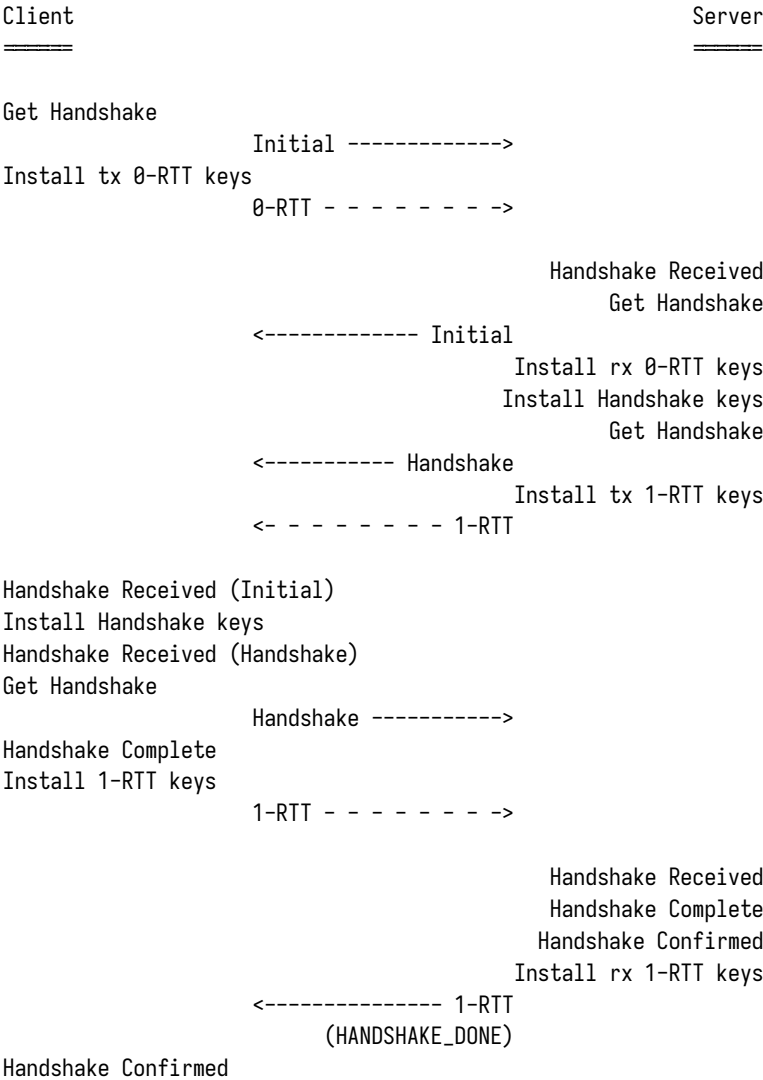


Figure 7: Interaction Summary between QUIC and TLS

6.3.4 Discarding Unused Keys

After QUIC has completed a move to a new encryption level, packet protection keys for previous encryption levels can be discarded. This occurs several times during the handshake, [as well as when keys are updated](#).

Packet protection keys are not discarded immediately when new keys are available. If packets from a lower encryption level contain CRYPTO frames, frames that retransmit that data **MUST** be sent at the same encryption level. Similarly, an endpoint generates acknowledgments for packets at the same encryption level as the packet being acknowledged. Thus, it is possible that keys for a lower encryption level are needed for a short time after keys for a newer encryption level are available.

An endpoint cannot discard keys for a given encryption level unless it has received all the cryptographic handshake messages from its peer at that encryption level and its peer has done the same. Different methods for determining this are provided for Initial keys and Handshake keys. These methods do not prevent packets from being received or sent at that encryption level because a peer might not have received all the acknowledgments necessary.

Though an endpoint might retain older keys, new data **MUST** be sent at the highest currently available encryption level. Only ACK frames and retransmissions of data in CRYPTO frames are sent at a previous encryption level. These packets **MAY** also include PADDING frames.

Packets protected with Initial secrets are not authenticated, meaning that an attacker could spoof packets with the intent to disrupt a connection. To limit these attacks, Initial packet protection keys are discarded more aggressively than other keys.

The successful use of Handshake packets indicates that no more Initial packets need to be exchanged, as these keys can only be produced after receiving all CRYPTO frames from Initial packets. Thus, a client **MUST** discard Initial keys when it first sends a Handshake packet and a server **MUST** discard Initial keys when it first successfully processes a Handshake packet. Endpoints **MUST NOT** send Initial packets after this point.

This results in abandoning loss recovery state for the Initial encryption level and ignoring any outstanding Initial packets.

An endpoint **MUST** discard its Handshake keys when the TLS handshake is confirmed.

[Check mini RFC: Packet Protection.](#)

7 TLS Error

If TLS experiences an error, it generates an appropriate alert as defined in Section 6 of [\[RFC 8446\]](#).

A TLS alert is converted into a QUIC connection error. The Alert-Description value is added to 0x0100 to produce a QUIC error code from the range reserved for CRYPTO_ERROR. The resulting value is sent in a QUIC CONNECTION_CLOSE frame of type 0x1c.

```

Handshake Packet {
  Header Form (1) = 1,
  Fixed Bit (1) = 1,
  Long Packet Type (2) = 2,
  Reserved Bits (2),
  Packet Number Length (2),
  Version (32),
  Destination Connection ID Length (8),
  Destination Connection ID (0..160),
  Source Connection ID Length (8),
  Source Connection ID (0..160),
  Length (i),
  Packet Number (8..32),
  Packet Payload (8..),
}

```

Figure 8: *Handshake Protected Packet*

QUIC is only able to convey an alert level of “fatal”. In TLS 1.3, the only existing uses for the “warning” level are to signal connection close; see Section 6.1 of [RFC 8446]. As QUIC provides alternative mechanisms for connection termination and the TLS connection is only closed if an error is encountered, a QUIC endpoint MUST treat any alert from TLS as if it were at the “fatal” level.

QUIC permits the use of a generic code in place of a specific error code. For TLS alerts, this includes replacing any alert with a generic alert, such as `handshake_failure` (0x0128 in QUIC). Endpoints MAY use a generic error code to avoid possibly exposing confidential information.

8 Packet and Frames

8.1 Packet

8.1.1 Handshake Packet

A Handshake packet uses long headers with a type value of 0x02, followed by the Length and Packet Number fields. The first byte contains the Reserved and Packet Number Length bits. It is used to carry cryptographic handshake messages and acknowledgments from the server and client.

Once a client has received a Handshake packet from a server, it uses Handshake packets to send subsequent cryptographic handshake messages and acknowledgments to the server.

The Destination Connection ID field in a Handshake packet contains a connection ID that is chosen by the recipient of the packet; the Source Connection ID includes the connection ID that the sender of the packet wishes

to use.

Handshake packets have their own packet number space, and thus the first Handshake packet sent by a server contains a packet number of 0.

The payload of this packet contains CRYPTO frames and could contain PING, PADDING, or ACK frames. Handshake packets MAY contain CONNECTION_CLOSE frames of type 0x1c. Endpoints MUST treat receipt of Handshake packets with other frames as a connection error of type PROTOCOL_VIOLATION.

Like Initial packets, data in CRYPTO frames for Handshake packets is discarded – and no longer retransmitted – when Handshake protection keys are discarded.

8.2 Frames

8.2.1 Crypto Frame

A CRYPTO frame (type=0x06) is used to transmit cryptographic handshake messages. It can be sent in all packet types except 0-RTT. The CRYPTO frame offers the cryptographic protocol an in-order stream of bytes. CRYPTO frames are functionally identical to STREAM frames, except that they do not bear a stream identifier; they are not flow controlled; and they do not carry markers for optional offset, optional length, and the end of the stream.

CRYPTO frames are formatted as shown in [Figure 9](#).

```
CRYPTO Frame {  
    Type (i) = 0x06,  
    Offset (i),  
    Length (i),  
    Crypto Data (..),  
}
```

Figure 9: CRYPTO Frame Format

CRYPTO frames contain the following fields:

- **Offset:** A variable-length integer specifying the byte offset in the stream for the data in this CRYPTO frame.
- **Length:** A variable-length integer specifying the length of the Crypto Data field in this CRYPTO frame.
- **Crypto Data:** The cryptographic message data.

There is a separate flow of cryptographic handshake data in each encryption level, each of which starts at an offset of 0. This implies that each encryption level is treated as a separate CRYPTO stream of data.

The largest offset delivered on a stream – the sum of the offset and data length – cannot exceed $2^{62} - 1$. Receipt of a frame that exceeds this limit

MUST be treated as a connection error of type `FRAME_ENCODING_ERROR` or `CRYPTO_BUFFER_EXCEEDED`.

Unlike `STREAM` frames, which include a stream ID indicating to which stream the data belongs, the `CRYPTO` frame carries data for a single stream per encryption level. The stream does not have an explicit end, so `CRYPTO` frames do not have a FIN bit.